IT SOLUTION DOCUMENT

Kickstream

AI stream agents for live football

The web studio, the match pipeline, the seven analyst agents in three teams, unified analytics, auto live, and the Azure resources defined in deploy/terraform, deployed from GitHub.

Kickstream does not implement Microsoft Marketplace SaaS fulfillment, an Entra purchase landing page, or a live Partner Center offer.

Challenge: The Synthetic Match Insights Engine for Premier League Studio.
Repository: https://github.com/Yash-Kavaiya/kickstream
Edition: 20 October 2026.

Companion briefing: kickstream-briefing.pdf. HTML entry: docs/solution/index.html. Scaling note: docs/architecture/scalable.html.

Contents

1 Solution overview and scope **1**
1.1 What changed in this edition . . . 1
1.2 The running demo . . . 1

2 Architecture and data flow **3**
2.1 Pipeline . . . 3
2.2 Agents and writers . . . 4
2.3 Where state lives . . . 4
2.4 How it grows . . . 5

3 Functional capabilities **5**
3.1 Studio and the program . . . 5
3.2 Destinations, broadcasts, reels, guests . . . 5
3.3 Analyst teams and the seven agents . . . 7
3.4 Unified analytics . . . 8
3.5 Ask Copilot, fans, auto live . . . 8

4 Security, privacy, and identity **9**

5 Deployment and operations **10**
5.1 From GitHub . . . 10
5.2 What Terraform creates . . . 11

6 Integrations and data sources **12**

7 Assumptions, constraints, and risks **13**
7.1 Assumptions . . . 13
7.2 Constraints . . . 13
7.3 Risks . . . 13

1 Solution overview and scope

Kickstream is live-streaming software for football. A host creates a broadcast, composes scenes in the studio, and goes live to YouTube, X, Facebook and Twitch at once. The picture on the program is a match event stream turned into live intelligence, AI narratives and recaps. Seven analyst agents, working in three teams, read the match as it happens. A unified analytics dashboard joins the match, the agents, the audience and the broadcasts on one page. Auto live runs a channel by itself with a spoken commentator in twenty languages. Agents then cut the recording's best moments into vertical reels.

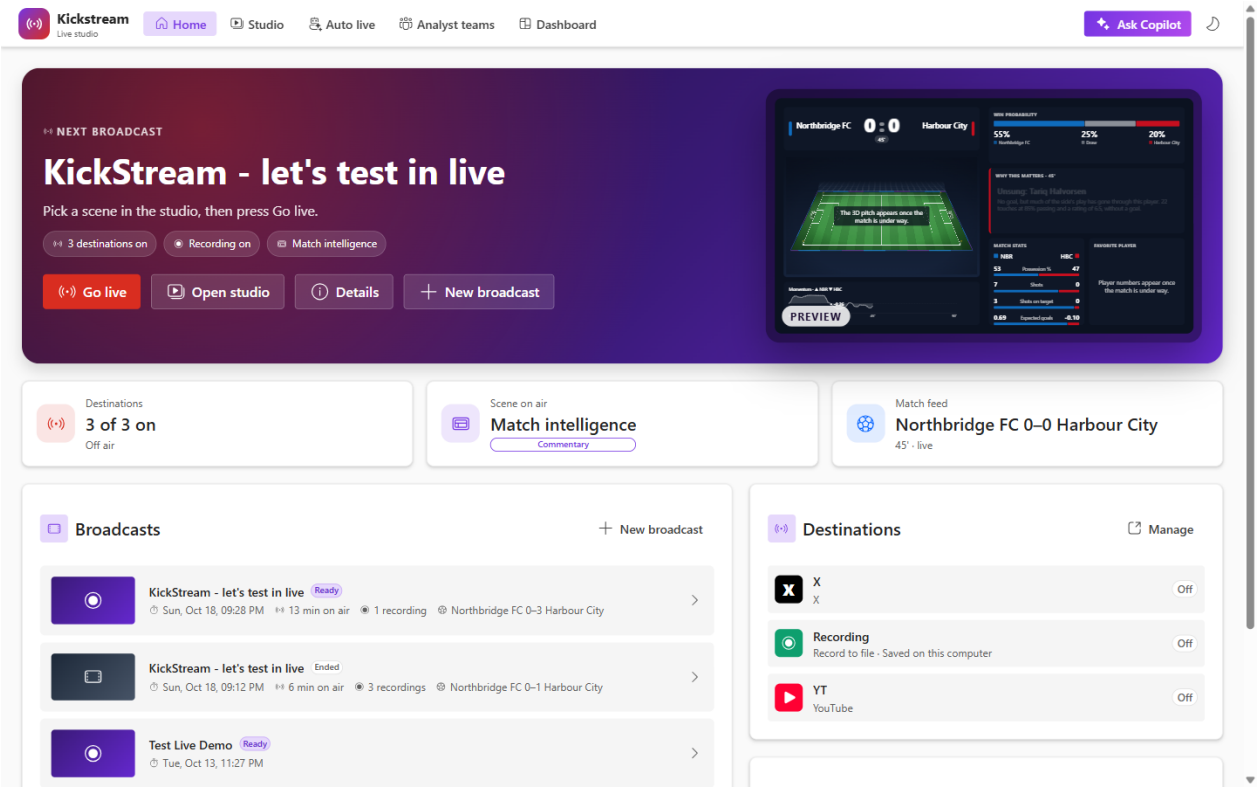
The match is synthetic by default: a seeded simulator plays Northbridge FC against Harbour City, two fictional clubs, so every insight can be produced and replayed without licensed data. This document describes the software in this repository: apps/web, services/pipeline, and the resources deploy/terraform creates. It is not a store listing.

What changed in this edition

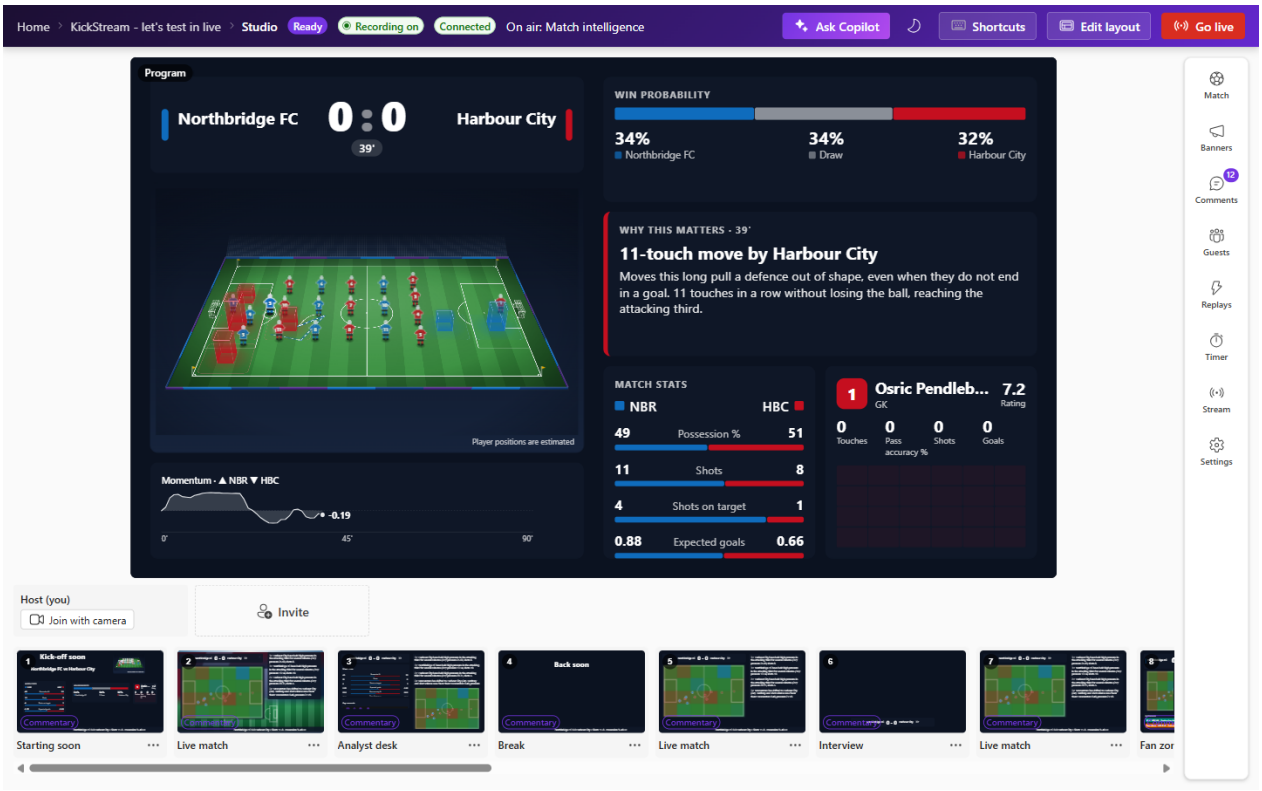
- **Teams of analyst agents.** The seven match agents now work as the Match Reading, Forecast and Story teams. /match is called Analyst teams.
- **Unified analytics.** /dashboard shows the score, odds, expected goals, momentum, team stats, each team's and agent's work, the best players, the audience, the broadcasts and auto live, live.
- **Agent tools.** Each agent takes custom instructions, up to three connectors of the operator's own, and a chat.
- **Auto live in twenty languages**, each with a matching Azure neural voice.
- **Studio.** One commentary for the program, nine league-style background plates, element delete, a large solo camera frame, and a goal celebration.
- **Operations.** Deploy from GitHub with OIDC, match logs in Cosmos DB, and narration through a Microsoft Foundry project.

The running demo

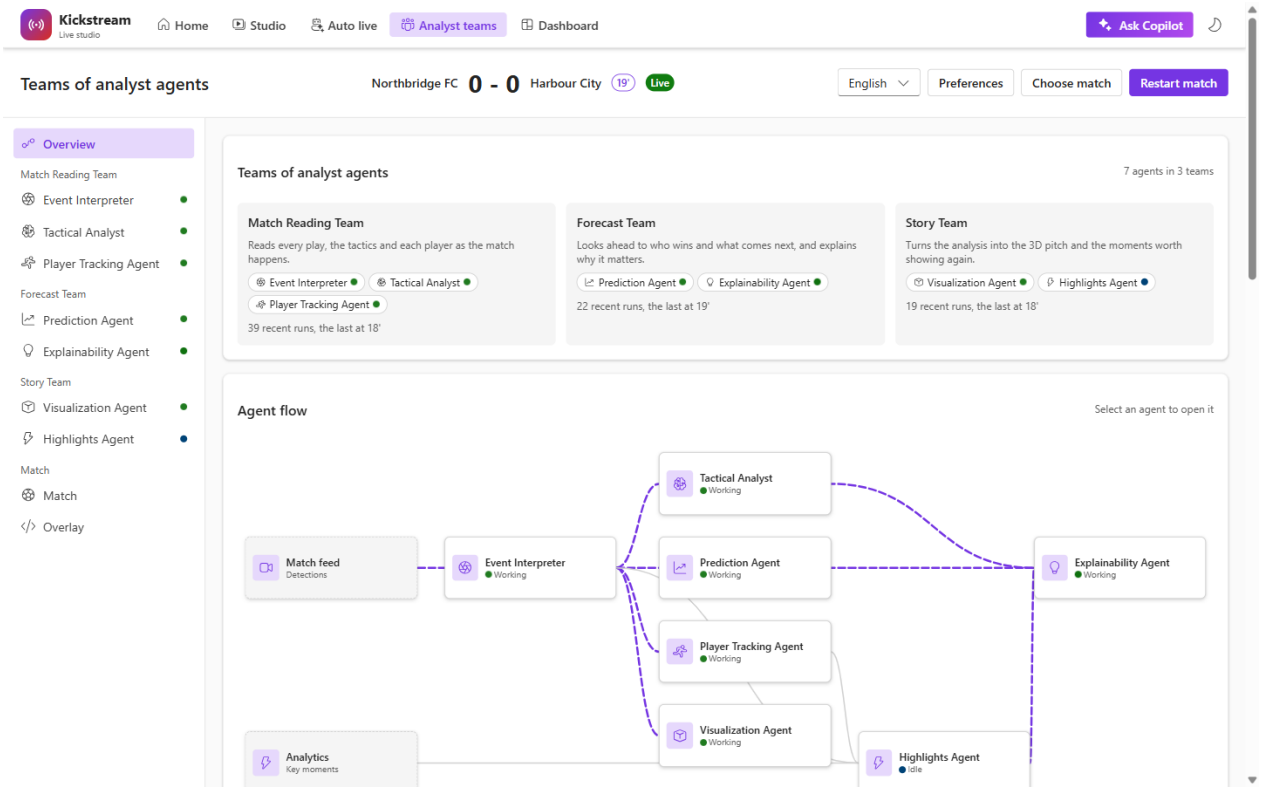
These pictures are the local demo on 20 October 2026: a synthetic match, the operator's home, the studio, the analyst teams, one agent, the dashboard, auto live, and a finished broadcast. The scene strip switches the program with a click. Keyboard shortcuts are not implemented.



Home. The next broadcast, a live program preview, the scene on air, the match feed, broadcasts and destinations.



Studio. The program at 39 minutes: score, 3D pitch, momentum, win probability, a “why this matters” card, match stats and a player card. The scene strip sits under the stage.



Analyst teams. Three team cards with their recent runs, and the agent flow from the match feed to the Explainability and Highlights agents.

- **In scope.** Home (/), a broadcast (/broadcasts/{id}), that broadcast's studio, /studio, Analyst teams (/match), unified analytics (/dashboard), a guest page (/guest/{secret}), and auto live (/live). The pipeline is Python 3.13 and FastAPI. The web app is Vite, React, TypeScript, and Fluent UI v9. Match-page languages are English, Spanish, and Hindi; auto live speaks twenty. Commentary modes are Fan and Analyst.
- **Out of scope.** A transactable Marketplace offer, fulfillment webhooks, metering, or a purchase landing page. StatsBomb Open Data inside a paid offer. A video detector: the synthetic match, or a JSON Lines file named by KICKSTREAM_DETECTOR_FILE, stands in for detections. Taking money from fans. Superchats are displayed; payment stays on the fan's platform.

The same boundaries are restated under Constraints so a reader who opens only that section still sees them.

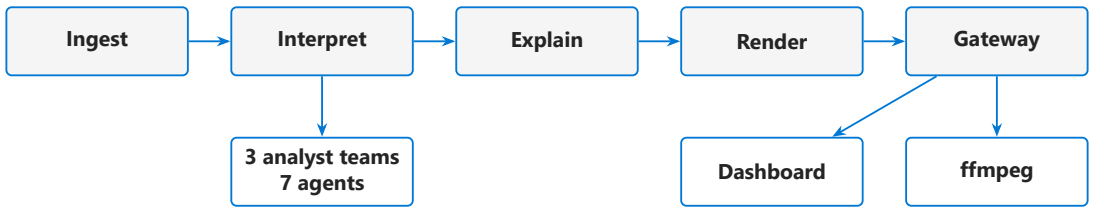
2 Architecture and data flow

Kickstream runs as one program. Locally that is FastAPI on port 8000 and the Vite app on port 5173. On Azure the template puts the server, which also serves the built web app, in a single Container App. Matches, the studio, and live streams are held in that process's memory, so the app is pinned to one replica (min_replicas = 1, max_replicas = 1).

Pipeline

A match moves through five stages. The seven match agents publish findings onto the same stream the browser reads. Stages talk only through the event bus and the contracts in packages/contracts, and never import one another.

Ingest	Interpret	Explain	Render	Personalize
Simulator, detector file, or a source	Analytics stats and pressure	Narration Fan and Analyst	Overlay scene widgets	Gateway mode and language

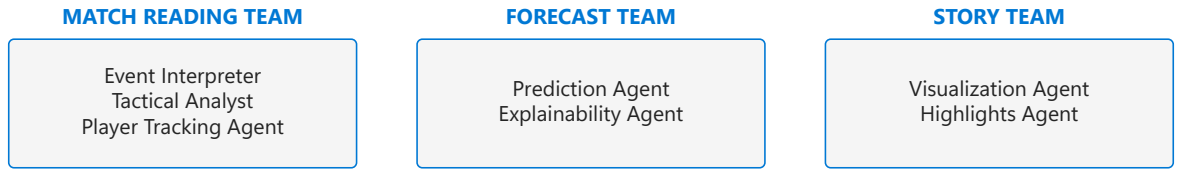


One encode. ffmpeg copies that picture to every destination. The dashboard reads the streams the gateway already sends.

The gateway fans the match out with Server-Sent Events and WebSockets. A late joiner is replayed the newest pitch scene and the newest player report, not the whole history. WebSockets are accepted only from the host the pipeline was reached at. Behind a proxy that rewrites Host, public names go in KICKSTREAM_ALLOWED_HOSTS. On the Container App, KICKSTREAM_PUBLIC_HOSTS is the app's own address.

Agents and writers

The seven match agents compute; one writes. They are declared the way a Copilot Studio agent is: instructions, triggers, tools, outputs and the agents they are connected to.



Narration, when a model is configured, is four Microsoft Agent Framework agents sharing one Foundry client: a Commentator and a Tactical Analyst on each key moment, a Recap Writer at half-time and full-time, and a Localizer into Spanish and Hindi. Each item is written once per mode and language and sent to every viewer on that variant, so model cost does not grow with the audience. Output is cached. If the model fails, that item falls back to a template and is marked scripted. Only the Explainability Agent, among the seven match agents, writes prose. That prose is English, at most 20 explanations a match. Agent chat and Ask Copilot word their answers from the same recorded facts, plus the agent's connectors when it has any.

Where state lives

- **Playback, studio, encode.** Process memory locally, and the same inside the one replica.
- **Broadcasts, studio document, uploads, cache, fans, auto live, agent switches and instructions, connectors, keys, recordings, reels.** Directory .data/ locally. Azure Files share data, mounted at /data.
- **Match log.** Memory unless Cosmos is configured. On Azure: database kickstream, container match_log, partition /match_id. Writes are queued and sent in the background, and a match not in memory is read back, so a restart keeps every replay.
- **Guest calls.** Communication Services, or KICKSTREAM_ROOMS=fake. The deploy creates the Communication Services resource.
- **Model.** A Foundry project, or an OpenAI-compatible endpoint, otherwise templates. Terraform does not create Foundry.

Going live opens /studio?output (1920 by 1080) in a hidden Chromium browser. ffmpeg encodes that picture once and copies it to every destination. Microsoft Edge is used when it is present. Auto live captures /live?output the same way.

How it grows

docs/architecture/scalable.html draws the shape to grow into: one match owner, one encode, and viewers that scale apart. A viewer gateway would scale with viewers, a studio API with hosts, a match worker would hold one lease per match, and an encode worker one ffmpeg per live show. That is a design note. It is not what deploy/terraform creates today.

3 Functional capabilities

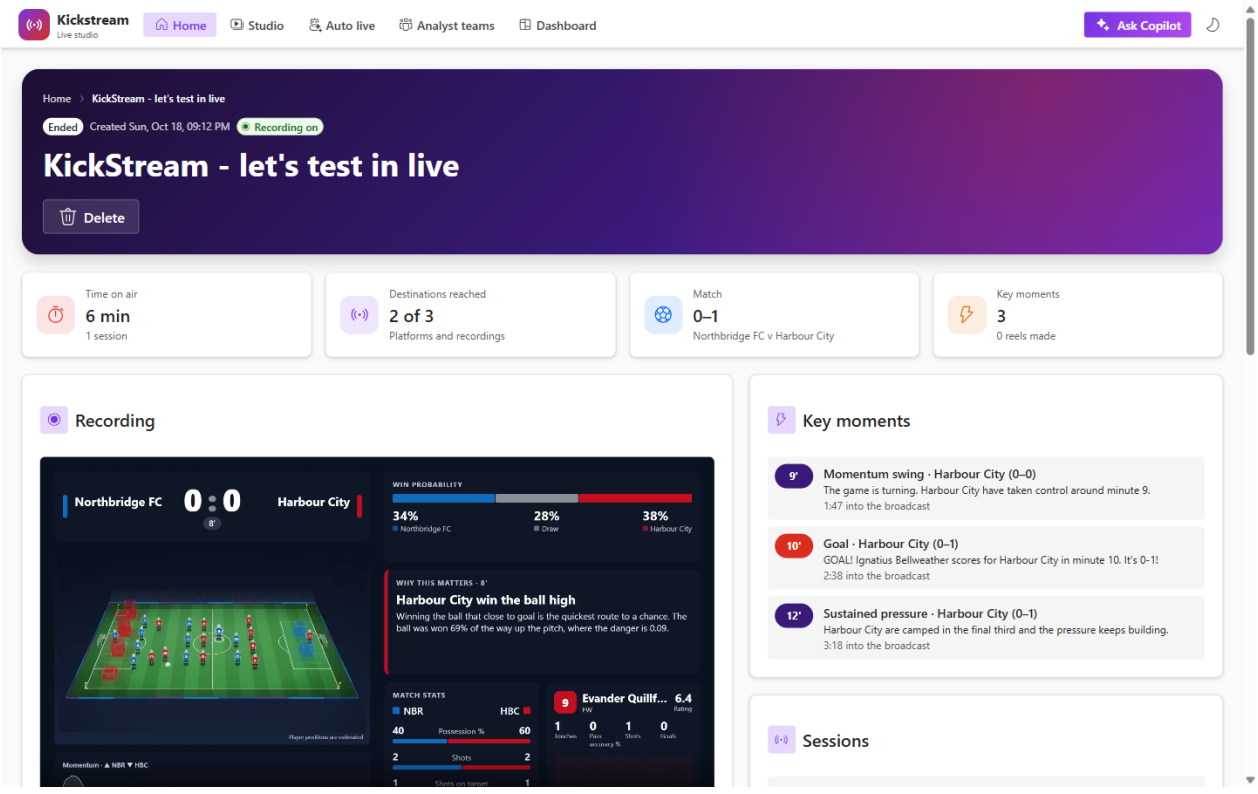
The path is Home, then a broadcast, then the studio. Scenes are one library shared by every broadcast.

Studio and the program

The top bar names the broadcast, the output state (Off air, Connecting, or LIVE with elapsed time), the scene on air, Edit layout, Ask Copilot, and the single Go live / End broadcast control. Tools, one at a time: Match, Banners, Comments, Guests, Replays, Timer, Stream, Settings. Templates: Starting soon, Live match, Analyst desk, Break, Full-time, Interview, Match intelligence, Fan zone, and 3D pitch. The program carries one commentary, Fan or Analyst, for the whole studio. Banners fire for goals and red cards, and a goal plays a short celebration on the program. Replays animate the 20 seconds before a key moment. Widgets cover score, charts, prediction, why it matters, 3D pitch, player card, comments, poll, superchats, and spotlight. Edit layout snaps, layers and deletes elements, and a scene can take one of nine original background plates (Night, Claret night, Sun gold, Red and black, Blue night, Navy gold, European night, Super League, Matchday pitch). These are broadcast looks, not a club's or a league's crest. The host can show a local camera without Azure; alone on the program it fills a large frame. The hidden browser receives the camera and microphone only, never the screen.

Destinations, broadcasts, reels, guests

The Stream tool sends the program to YouTube, X, Facebook and Twitch. Record to file writes an MP4 and needs no key. Each destination shows connecting, live, retrying, or failed. One failure does not stop the others. The only sound is the guest call. With nobody unmuted, the stream is silent.

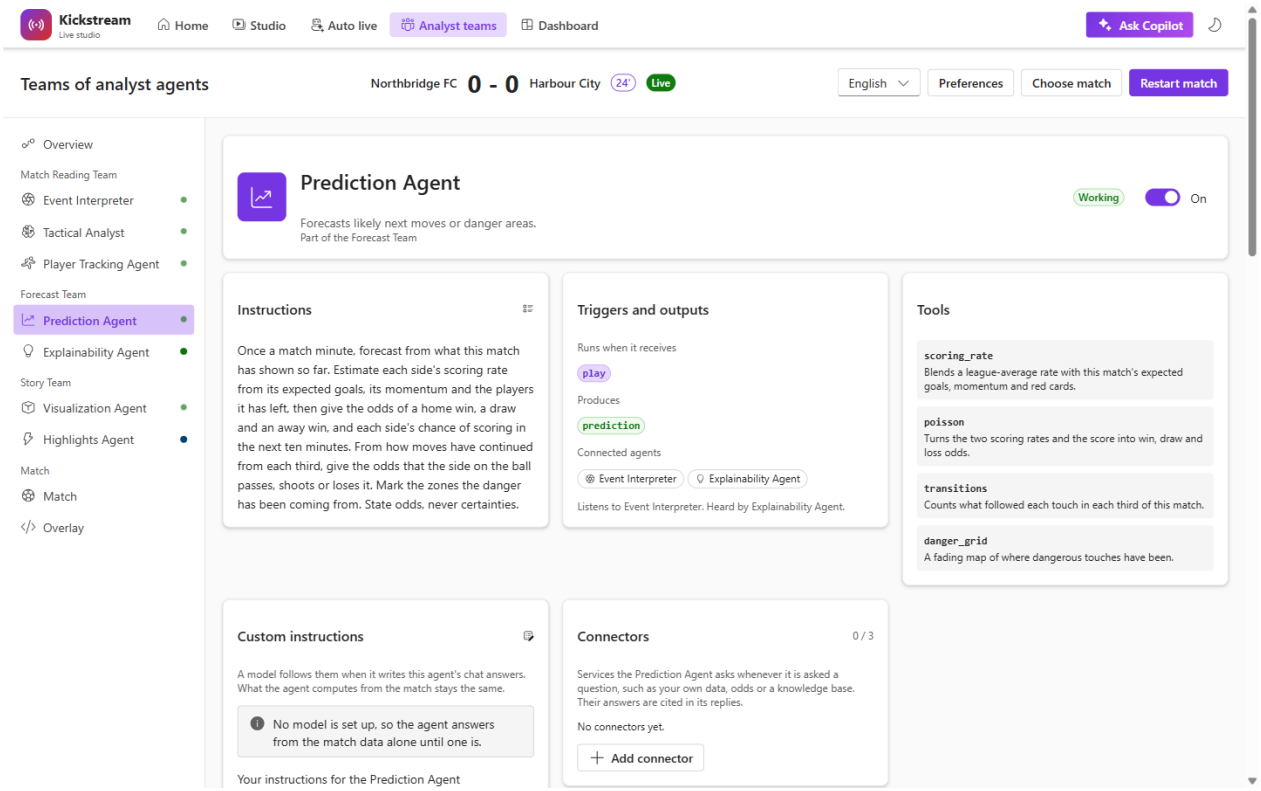


A finished broadcast: time on air, destinations reached, the score, key moments with their commentary, recordings, and each session's destinations.

End broadcast keeps sessions, recordings, time on air, destinations, the score, and key moments. Generate reels cuts up to four 15-second vertical clips (1080 by 1920). Each clip starts 8 seconds before its moment. A Highlight Editor and a Social Copywriter run when a model is set; they do not choose the edit points. A reel can be edited in Clipchamp and the export added back beside the original.

Guest roles are Speaker, Co-host, and Member. Speaker and Co-host can be placed on stage; a Member cannot. A Co-host can go live, end or delete the broadcast, and make or revoke invites. Up to four people are on stage. Calls use Azure Communication Services Rooms.

Analyst teams and the seven agents



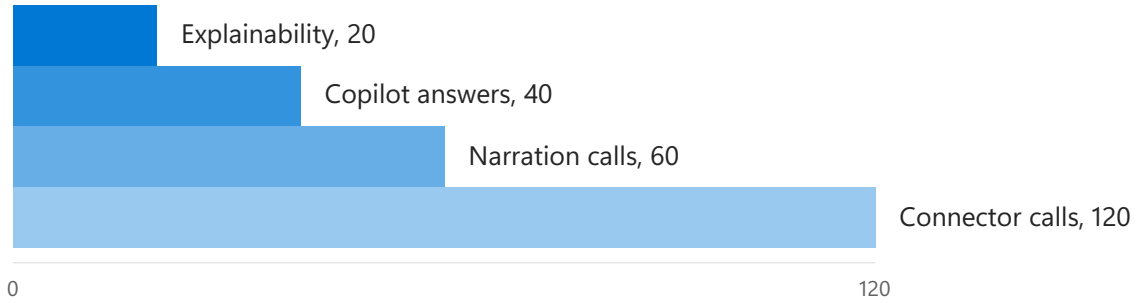
One agent, the Prediction Agent of the Forecast Team: its instructions, triggers and outputs, tools, custom instructions and connectors.

Team	Agent	Output
Match Reading	Event Interpreter	Third, flank, phase, possession sequence, danger
Match Reading	Tactical Analyst	Once a minute: pressure, tempo, tilt, patterns, control or chaos
Match Reading	Player Tracking Agent	Touches, passing, shots, expected goals, rating, heat map, distance, top speed
Forecast	Prediction Agent	Win, draw, loss, goal chance in ten minutes, next action, danger grid
Forecast	Explainability Agent	English "why this matters" when patterns change, odds move by 12 points, or a hidden event appears
Story	Visualization Agent	3D pitch. Only the ball and the player on it are known positions
Story	Highlights Agent	Ranked moments and hidden events

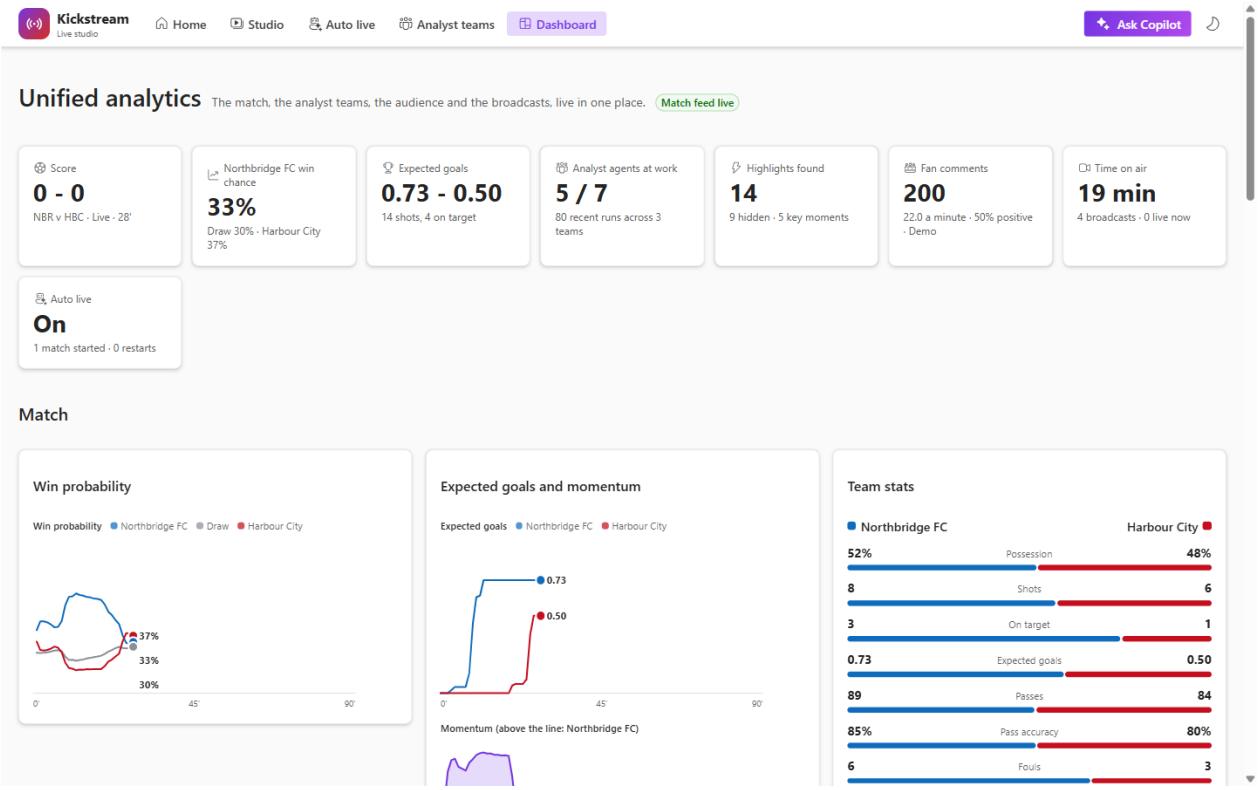
Each agent can be switched off; the agents that listen to it then have less to work on. For the studio's users, each agent also has:

- **Custom instructions**, up to 2,000 characters. A model follows them, after its own rules, when it writes as that agent. What the agent computes does not change.
- **Connectors**, up to three per agent. When the agent is asked a question, Kickstream posts the question as JSON to each connector that is on, waits up to 10 seconds, and cites the answer as a Connector source. Answers are given to the model as facts, never as instructions. At most 120 connector calls a match.
- **Chat**, answered from that agent's part of the match and its connectors. Anyone who can open /match can chat.

The chart counts calls the product allows in one match, not a measured audience.



Unified analytics



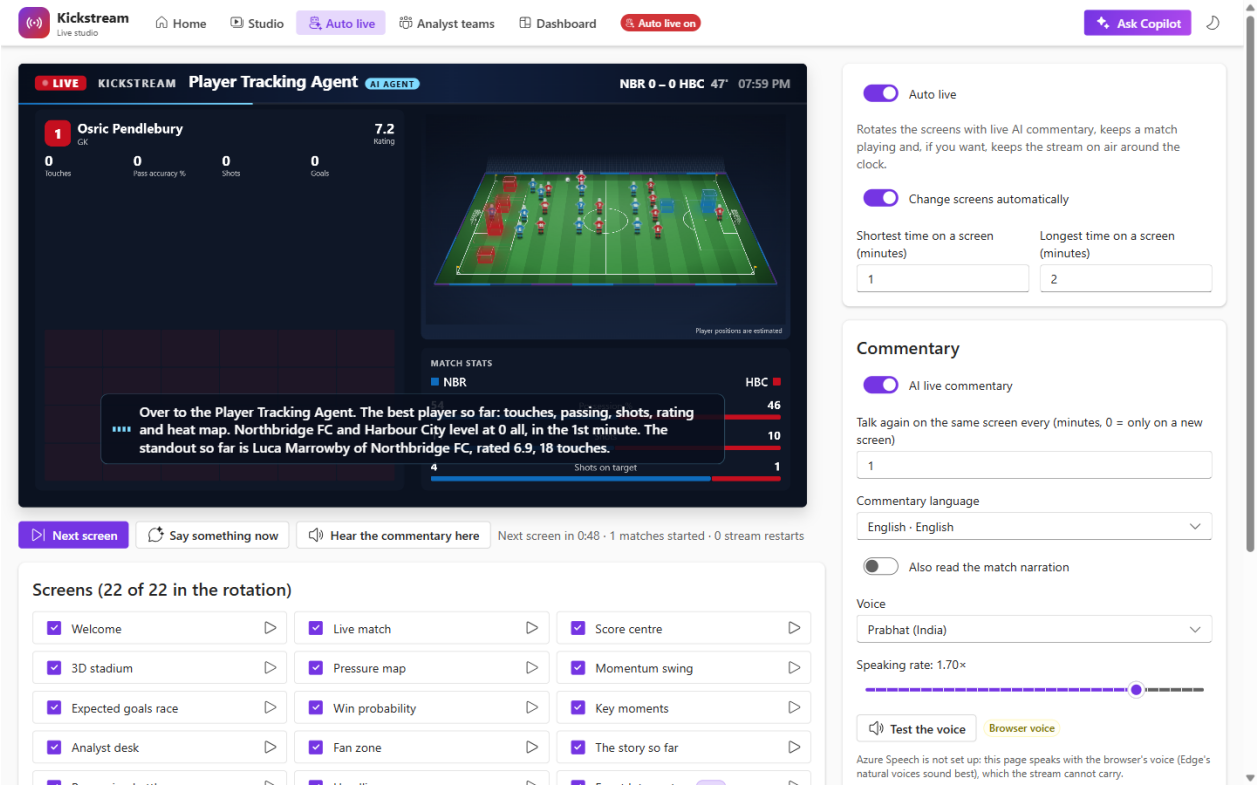
Unified analytics at 28 minutes: score, win chance, expected goals, analyst agents at work, highlights found, fan comments, time on air and auto live, above the win probability, expected goals and momentum, and team stats.

/dashboard is one live page built from the streams the server already sends: the score and odds, expected goals and momentum, team stats and tactics, work by team and every agent's runs, highlights by kind, the best-rated players, the audience's mood, platforms and topics, output and recent broadcasts, and auto live. It is a studio page, behind sign-in on a public deployment.

Ask Copilot, fans, auto live

Ask Copilot answers from the live match and from the newest 50 finished matches in .data/matches.json. Cards never come from the model. Web search is off until enabled: Wikipedia, or Tavily with KICKSTREAM_TAVILY_API_KEY. Typing @ sends the question to one agent. The same read-only answers are MCP tools at POST /mcp.

The fan zone starts on a demo feed. X is optional. Other platforms post to POST /api/fans/ingest. Kickstream shows superchats and takes no payment.



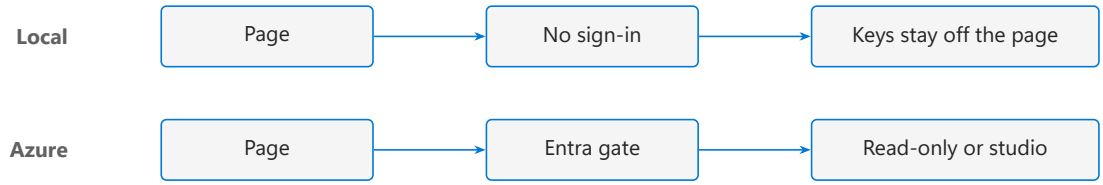
Auto live on: the Player Tracking Agent screen with a spoken line on screen, the 22 screens in the rotation, and the commentary language and voice.

Auto live (/live) cycles 22 scenes, including one per match agent and one of all seven at work, on a random gap the operator sets (5 to 10 minutes by default). An AI commentator introduces each scene and speaks again as often as set. Commentary speaks English, Spanish, Hindi, Portuguese, French, German, Italian, Dutch, Polish, Turkish, Arabic, Russian, Chinese, Japanese, Korean, Indonesian, Bengali, Tamil, Urdu or Marathi, each with matching Azure neural voices. Screen labels exist in English, Spanish and Hindi. Stream 24/7 publishes /live?output and restarts it if it drops. It waits while the studio is on air.

4 Security, privacy, and identity

On a developer machine the application has no sign-in of its own. Anyone who can reach the pipeline can stream to the configured channels, so the API stays on that machine. Stream keys live in .data/output-keys.json and are never sent back to a page. A guest tunnel must be listed in KICKSTREAM_PUBLIC_HOSTS first. If that list is empty, the gate is off and the tunnel would open the whole studio.

On Azure, Container Apps authentication signs studio users in with Microsoft Entra ID. The application registration is named “Kickstream studio” plus the resource name. sign_in_audience is AzureADMyOrg. The only Graph permission is User.Read. App-role assignment is required. The deployer and every object id in studio_users receive default access. The app reads KICKSTREAM_SIGNED_IN_HEADER, set to x-ms-client-principal-id. When public_view is true, anonymous clients reach the app and the app keeps /live and /match read-only, sending studio pages, the dashboard included, to /.auth/login/aad. When it is false, anonymous clients are redirected to login. /guest/*, /api/guest/*, and /assets/* stay open either way.



AzureADMyOrg. Public view keeps /live and /match read-only. Studio pages and /dashboard go to sign-in.

Only the studio's users can change an agent's instructions and connectors. A connector key is sent in the header the operator names, or as a bearer token, and is kept in the owner-only .data/agent-secrets.json, never shown again. Connector addresses are readable by anyone who can read /api/agents, so a key belongs in the key field, never in the address. Redirects are not followed. Connector answers reach the model as facts, never as instructions.

Cosmos DB has local_authentication_enabled = false. The Container App's system-assigned identity holds Cosmos DB Built-in Data Contributor on that account only. Foundry is not created by Terraform. If foundry_resource_id is set, the same identity receives Azure AI User on that existing resource, and signs in to Foundry with Entra ID rather than a key.

Deploying from GitHub signs in through OIDC with no secret. The Entra app GitHub Actions uses is trusted only for this repository's production environment. It is an Owner of the subscription and holds two Microsoft Graph permissions, Application.ReadWrite.OwnedBy and AppRoleAssignment.ReadWrite.All, so Terraform can register the studio's sign-in. Terraform state sits in a private storage account used with Entra ID.

Container App secrets are the registry admin password, the Communication Services connection string, the Entra client secret, the Speech key, and, only when provided, the API-Football key and the LLM key. The file share is Standard LRS with TLS 1.2 minimum, mounted with the storage account access key. The registry is Basic with the admin user enabled.

Fan text is truncated and shown as plain text. MCP on a deployment sits behind the studio sign-in and only reads. A co-host can end or delete the broadcast and can go live. Synthetic clubs are fictional. Favourite-club preferences stay in the browser.

5 Deployment and operations

deploy/deploy.ps1 and deploy/deploy.sh require the Azure CLI and Terraform 1.9 or newer. They set ARM_SUBSCRIPTION_ID from az account show, apply the container registry first, build kickstream:<tag>, then apply the rest with image_tag. Build uses az acr build, or local Docker with -LocalBuild or –local-build. deploy/destroy.ps1 asks before deleting the stack and the app registration unless -Force is passed. deploy/destroy.sh is the shell counterpart.

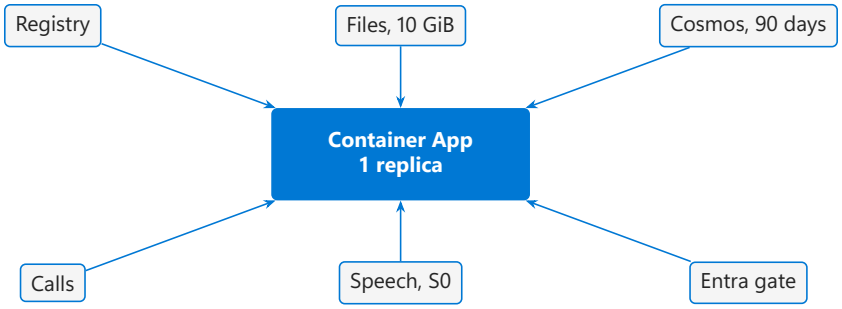
From GitHub

.github/workflows/ holds two workflows. **CI** runs on every push and pull request: ruff, the pipeline tests, a check that both generated contract files match the schema, the web typecheck, tests and build, and terraform validate. **Deploy** runs after CI passes on main, or from the Actions tab, does what deploy.sh does, and waits until the new revision answers. It stays off until deploy/github-setup.ps1 (or .sh) has created the OIDC app, the state storage account, and the repository variables AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID, TFSTATE_RESOURCE_GROUP, TFSTATE_STORAGE_ACCOUNT and STUDIO_USERS. LLM_API_KEY and API_FOOTBALL_KEY are repository secrets.



What Terraform creates

Providers: azurerm approximately 4.0, azuread approximately 3.0, azapi approximately 2.0, random approximately 3.6. Default region is centralindia. Communication Services data location defaults to India.



Terraform creates these resources and no others. Foundry is not one of them.

Registered namespaces: Microsoft.App, Microsoft.CognitiveServices, Microsoft.Communication, Microsoft.ContainerRegistry, Microsoft.DocumentDB, Microsoft.OperationalInsights, Microsoft.Storage.

Resource	What the template sets
Resource group rg-${name}	Deletion is not blocked while resources remain
Log Analytics	PerGB2018, 30-day retention, permanently deleted on destroy
Container Registry	Basic, admin user enabled
Storage and share data	Standard LRS, TLS 1.2, 10 GiB, mounted at /data
Communication Services	Guest rooms
Azure AI Speech	Kind SpeechServices, SKU F0 or S0, default S0
Cosmos DB	Serverless, session, keys off, container match_log, TTL default 90 days
Container Apps environment	cae-${name}, attached to Log Analytics
Container App	External ingress port 8000, one replica, 2 vCPU, 4 GiB
Entra auth config current	Studio gate. Not a Marketplace purchase app
Azure AI User	Only when foundry_resource_id is set

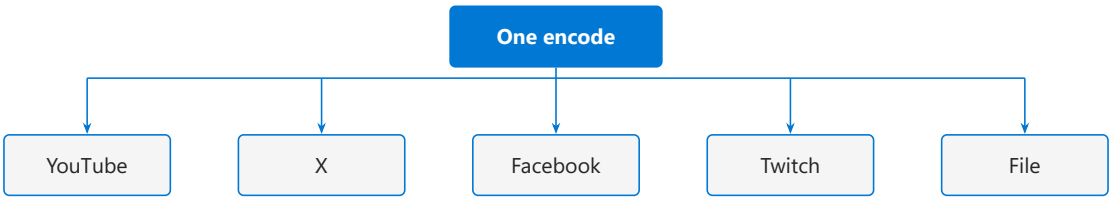
The container size is 2 vCPU and 4 GiB because the hidden browser and ffmpeg need at least two vCPU, and Container Apps pairs 2 GiB with each vCPU. Both probes call /api/health on port 8000, past the sign-in. Startup runs every 10 seconds and allows 30 failures. Liveness waits 30 seconds, then every 30 seconds, and allows 6 failures. The payload reports the match, the stream state, and whether auto live is on.

Defaults in variables, overridable from terraform.tfvars or repository variables: autolive true, public_view true, real_sources false, speech_sku S0, speech_daily_chars 200000, match_log_ttl_days 90. Watch Log Analytics (30 days), /api/health, the per-destination status in the studio, the dashboard's output and auto live cards, the file share (Record to file plus Stream 24/7 fills 10 GiB), and the speech character cap (past it, captured auto-live speech goes quiet until midnight UTC).

6 Integrations and data sources

Kickstream encodes one program and copies it. It does not host the public video.

Destination	Host supplies
YouTube, Facebook, Twitch	A stream key. The ingest address is filled in
X	The account's server address and stream key
Record to file	Nothing. An MP4 is written on the data share



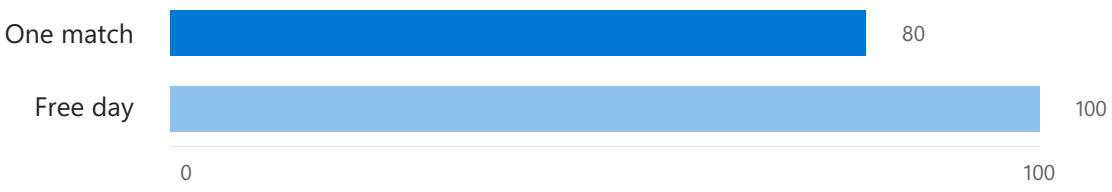
ffmpeg copies that picture. One failure does not stop the others.

Do not run the pipeline with –reload while going live: on Windows that mode cannot start ffmpeg.

A deploy offers the synthetic match only unless real_sources = true. Live feeds have key events and no ball positions. Pitch widgets then show that the coordinate is not available, rather than an invented one.

Source	Licence and limit
Synthetic match	Fictional clubs. Same seed, same match. Default on a deploy
Detector file	KICKSTREAM_DETECTOR_FILE. No video decoder in Kickstream
StatsBomb Open Data	StatsBomb Open Data is non-commercial and must not ship in a paid offer. Attribution: logo and "Data: StatsBomb"
API-Football	Free key, about 100 requests a day. One match is on the order of 80 requests
OpenLigaDB	No key, ODbL, credit "Data: OpenLigaDB". Clock estimated from kick-off. Default leagues bl1,la1,ucl

The bars are API-Football requests. They are provider limits, not a measured audience.



Azure AI Speech is the voice the capture can hear, in twenty languages. F0 allows 0.5 million characters a month; S0 bills per character. The daily cap defaults to 200000. Microsoft Foundry is used when both the project endpoint and the deployment name are set, signed in with Entra ID; otherwise KICKSTREAM_LLM_BASE_URL, KICKSTREAM_LLM_MODEL, and KICKSTREAM_LLM_API_KEY. KICKSTREAM_LLM_MAX_CALLS defaults to 60 narration calls a match. Agent connectors are the operator's own HTTP services: Kickstream posts question, agent, agent_name, match_id and source as JSON and reads a JSON answer field or plain text. Copilot web search uses Wikipedia, or Tavily when its key is set. The X and Tavily adapters are tested against documented payloads, not against those live services. Guest rooms use Communication Services, or KICKSTREAM_ROOMS=fake locally. Reels can be finished in Clipchamp, which takes the reel as a file.

7 Assumptions, constraints, and risks

Assumptions

The deploying operator can create the resource group, the Entra application, and the role assignments; deploying from GitHub needs a subscription Owner and a Global Administrator to grant the Graph permissions. The image can run ffmpeg and Chromium. Synthetic clubs are fictional, and a head-to-head is only matches this installation has finished. Viewers share narration; personalization is a gateway filter. A model is optional. Templates are the supported path when no endpoint is set. Connectors belong to the operator and are trusted to answer, not to instruct.

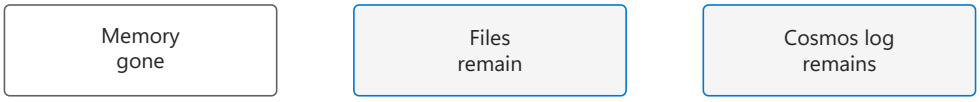
Constraints

StatsBomb Open Data is non-commercial and must not ship in a paid offer. Kickstream does not implement Microsoft Marketplace SaaS fulfillment, an Entra purchase landing page, or a live Partner Center offer.

One replica only. Live feeds do not include ball positions. API-Football's free plan is about 100 requests a day. OpenLigaDB can arrive late, and its clock is estimated. Explainability and Ask Copilot are English; auto live speaks twenty languages but labels its screens in three. Keyboard shortcuts are not implemented. The stream has no bed and no crowd. Reel in-points are fixed at 8 seconds before the moment, for 15 seconds. The current templates pull the image with the registry admin user and mount files with the storage account key.

Risks

A replica restart drops the in-memory match and the ffmpeg process. The file share and the Cosmos log remain.



Auto live can restart a 24/7 stream from saved settings. A studio encode does not resume itself. The default public view lets anyone watch /live and /match read-only, and chat with the agents there. An empty public-host list plus a tunnel exposes studio controls. A co-host invite is an operator credential. The GitHub deploy identity is a subscription Owner; protect the production environment and the main branch accordingly. A connector address is visible through /api/agents; a key placed in it leaks. S0 speech spends money per character, and the daily cap makes the captured voice go quiet rather than switching the encode to the browser voice. Record to file with Stream 24/7 fills the 10 GiB share. Cosmos has a single region. One failed destination does not stop the others, and nothing fails over to a second region.

The safe commercial reading is: operate the synthetic match, or a feed you have a right to use; keep StatsBomb off any paid offer; and do not describe a Marketplace purchase flow as part of the product that ships today.